The subsequent `Cookie:` request header would then appear as `Cookie: flavour[best]=choc%20chip%20obvs; flavour[worst]=there%20is%20no%20worst` and would result in the following data appearing in `$_COOKIE`:

```
array(1) {
  ["flavour"]=>
  array(2) {
    ["best"]=>
    string(14) "choc chip obvs"
    ["worst"]=>
    string(17) "there is no worst"
  }
}
```

That said, it is not unusual to see structured cookie data encoded as JSON. There is some haziness surrounding the support for structured data in the specification – RFC6265, sections 4 and 5 https://datatracker.ietf.org/doc/html/rfc6265 – and using a JSON string neatly steps around the problem.

## Query String Data

If URL routing and headers don't meet your needs for data passing, we have some additional mechanisms to look at. We've already touched upon query strings above, which allow arbitrary data to be sent from client to server during requests. The request data (aka "payload") itself has some handy utility wrappers; PHP exposes request data in a couple of easy-to-use ways.

Query strings operate on a `key=value` basis and are placed after the resource path in the URL (delimited by a `?` character), with each key/value pair delimited by a `&` character. PHP has a number of helper functions for parsing and constructing these URL components; we'll focus on the parsing here. Create a new `index.php` and serve as before; the contents simply need to be as follows:

```
// index.php
var_dump($_GET);
```

```
$ curl 'http://localhost:8080/index.php?foo=bar&cow=moo'
array(2) {
  ["foo"]=>
```